Can a Decision Model Be the AI Inside a Database?

An experimental study of Jev on DuckDB

7 October 2026

Summary

Jev (TypeSafe AI, released 15 September 2026) is a non-generative *decision model*. It takes a piece of state and a set of typed questions (yes/no, single choice, ordinal score) and returns the answers with probabilities in one pass, at $0.042 per million input tokens. This report asks whether it can usefully be combined with a database, and tests five claims that have circulated since its release. Each claim was given a pass/fail criterion before the corresponding data were collected.

Claim	Criterion fixed in advance	Result
Jev can execute semantic SQL at LLM quality	Mean quality no more than 0.1 below published systems (SemBench, movie scenario)	**Supported.** 0.765, against 0.689–0.741 published and 0.745 for a cheap LLM in the same harness
It does so far more cheaply	Measured, no threshold	**3× cheaper and 5× faster** than a cheap LLM in the same harness, not the 100× of the announcements
"Jev first, LLM when unsure" gives LLM quality at low cost	Escalating 30% of rows recovers at least 60% of the accuracy gap	**Task-dependent.** Holds for record matching (two data sets); fails for sentiment and aspect predicates
Jev's probabilities make semantic aggregates cheap	At least 3× fewer labels than sampling; 95% intervals cover at least 90%	**Not met.** 2.6× and 94.5%; an open model's token probabilities do equally well
Jev can give the optimizer selectivities for ordering semantic predicates	Median q-error at most 2; an LLM at least 2 points better than Jev to escalate to	**Not worth building.** Best and worst orders differ by 11–27%; packing the predicates into one request matches the best order

The two largest gains measured do not depend on the model at all: rewriting a pairwise semantic join as "judge each row once, then join relationally" (8,128 model calls reduced to 128, with a higher F1), and packing all predicates on a row into one request (20× less wall time for a 21-attribute profile).

Bottom line. Jev is currently the cheapest and fastest way to get a per-row probability inside SQL, and that is the right way to think of it. It is not more accurate than a cheap LLM, and it offers no capability that an open-weight model's token probabilities lack.

1 Questions and prior work

The study started from three questions: can Jev's strengths be combined with a database; can the "AI" in AI-for-databases be Jev; and are the widely repeated claims about it (above all, that *semantic-operator cascades* are its most valuable use) true?

Two pieces of prior work frame the answers. JEVDB [1], posted a week before this study, builds semantic filter, join, classification and ranking operators on Jev and reports 89× lower latency and 13.5× lower cost than LOTUS [3] on SemBench [2]. It lists selectivity estimation, predicate ordering, calibration analysis and open-weight alternatives as future work; those are the parts tested here. My own earlier experiments had shown that Jev cannot choose query plans zero-shot (AUC 0.33–0.52 on the Join Order Benchmark), that it matches a supervised matcher on product data, and that a Jev-then-Opus cascade reached Opus quality at 19% of the cost on one product-matching data set.

2 Setup

System. About 500 lines of Python register Jev as vectorised functions in DuckDB 1.5.6: `jev_noul` (probability of yes), `jev_choice`, `jev_score` and `jev_nouls` (several yes/no questions in one request). Each batch is de-duplicated, sent concurrently, cached in SQLite by (model and configuration, state, question) and metered. The same functions run against an LLM by swapping the backend, and a cascade variant escalates unsure rows to a second backend.

Models. Jev 1.13.0 through the official API. LLMs through OpenRouter with the provider pinned on every request, because unpinned requests to one model landed on five different providers in six calls. The LLM used as the upper tier was chosen by a rule written before the candidates ran: on 637 product pairs where Jev is unsure, take the cheapest of eight routes within 3 points of Opus 5.5 (86.7%). Only GPT 6 Luna with low reasoning qualified (84.5%; Jev itself 74.9%). DeepSeek V4.1 Flash, DeepSeek V4 Pro and Qwen 3.8 Flash were run with thinking off to read token probabilities, and Opus 5.5 served as a stronger upper tier in the last experiment.

Data. (i) SemBench movie scenario [2]: 2,000 reviews with its ten queries, plus a 16,000-review set drawn by its own generator (15,388 distinct reviews). The truth is the critic's verdict. (ii) MAMS [4]: 3,949 restaurant-review sentences; 7 aspects × {mentioned, negative, positive} give 21 predicates with selectivities from 1% to 73%. (iii) Four record-matching test sets [5] and two end-to-end matching sets from the earlier work.

Protocol. Prompts are first drafts written from the task descriptions and were never edited after seeing labels. Primary analyses, success criteria and stop rules were written down, with checksums of the scripts, before each run (`experiments/database/protocol/` in the repository). Timings come from cold runs; every judgment is cached once and all later analyses replay the cache.

3 Results

3.1 Semantic SQL on DuckDB

All ten SemBench movie queries run as plain SQL. Table 1 gives quality by query, scored with SemBench's own evaluator; Figure 1 gives totals. Jev was run cold three times with almost no variation between runs.

Table 1. SemBench movie scenario, 2,000 reviews. F1 and correlation: higher is better. Relative error: lower is better.

Query	Metric	Jev	LLM	Cascade	Published (4 systems)
Q1 five positive reviews	F1	1.000	1.000	1.000	0.952–1.000
Q2 five positive, one movie	F1	0.600	0.600	0.600	0.923–1.000
Q3 count positive	rel. error	0.214	0.214	0.143	0.357–0.571
Q4 positive ratio	rel. error	0.214	0.143	0.143	0.357–0.571
Q5 ten same-sentiment pairs	F1	0.900	0.800	0.800	0.316–1.000
Q6 ten opposite pairs	F1	0.600	0.400	0.400	0.632–0.850
Q7 all opposite pairs	F1	0.732	0.769	0.772	0.202–0.695
Q8 count by sentiment	rel. error	0.040	0.000	0.040	0.081–0.323
Q9 score each review	Spearman	0.795	0.763	n/a	0.748–0.780
Q10 rank movies	Spearman	0.412	0.421	n/a	0.404–0.438

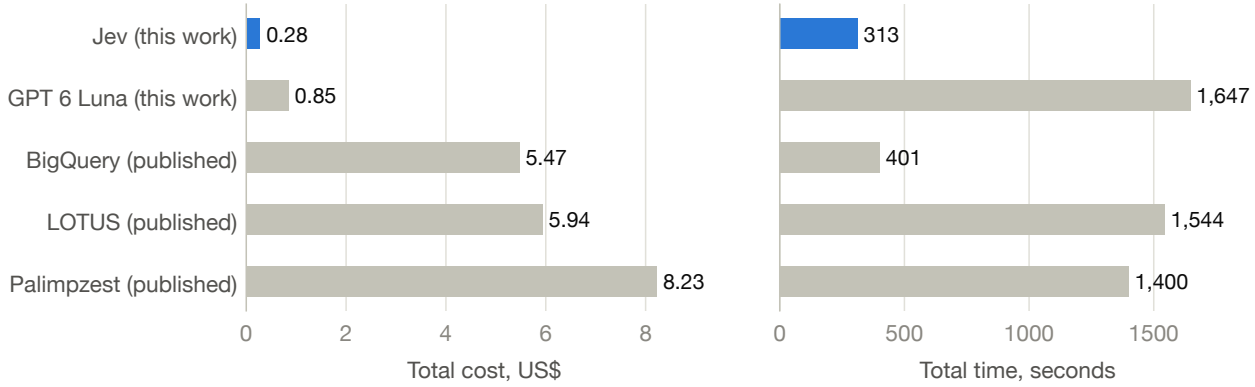


Figure 1. Total cost and time for the ten queries. Published figures were measured in 2025 with Gemini 2.5 Flash on other infrastructure, so only the first two rows are a like-for-like comparison.

Mean quality (SemBench's aggregation) is 0.765 for Jev, 0.745 for the LLM, and 0.741, 0.689 and 0.721 for BigQuery, LOTUS and Palimpzest. Against the LLM in the same harness Jev is 3.0× cheaper and 5.3× faster. Two rewrites mattered more than the choice of model. Q5–Q7 ask for pairs of reviews; judging each of the 128 distinct reviews once and joining relationally replaces 8,128 pairwise calls and raises F1 on Q7 from 0.73 to 0.79, bringing all ten queries to 90 seconds and $0.07. Returning the k most probable rows instead of the first k found (a variant added after seeing Q2) gives F1 1.0 on all four LIMIT queries.

3.2 When does a cascade work?

This is the claim the study set out to exploit, and the answer is the main finding. A cascade needs two things: an upper tier that is clearly more accurate than Jev, and Jev errors that sit in the rows it is unsure about. Record matching has both. The movie data lack the first, the restaurant data lack the second.

Table 2. Cascades by data set. "Recovered" is the share of the Jev-to-upper-tier accuracy gap regained by escalating the least-sure 30% of rows (random choice recovers 30%; the criterion was 60%). Brackets are 95% bootstrap intervals.

Data	Upper tier	Upper tier vs. Jev (accuracy)	Outcome of the cascade
Product matching, Abt-Buy (earlier work)	Opus 5.5	+11.8 points on Jev's unsure rows	Works: F1 0.902 to 0.937, 12% of rows escalated
Product matching, Walmart-Amazon	Opus 5.5	+7.5 points on Jev's unsure rows	Works: F1 0.884 to 0.929, +0.045 [0.016, 0.074], 10% escalated
Movie reviews (15,388)	GPT 6 Luna	−1.5 points overall	Fails: 18–63% of rows escalated for no gain
Movie reviews (1,851)	Opus 5.5	+1.0 point overall [−0.1, 1.9]	Nothing to gain: below the 2-point bar
Restaurant reviews (546 × 21)	GPT 6 Luna	+4.0 points [3.4, 4.5]	Fails: recovered 38% [32, 46]
Restaurant reviews (546 × 21)	Opus 5.5	+3.3 points [2.7, 3.9]	Fails: recovered 36% [28, 46]

On the movie data even Opus is not meaningfully better than Jev against the human labels, so there is nothing for a cascade to gain; the labels cap every model. On the restaurant data stronger models are better by 3–4 points, yet Jev's probabilities do not locate its mistakes. Figure 2 shows why, without involving any upper tier: on matching data 91–100% of Jev's errors lie in its least-sure 30% of rows, against 80% for movie reviews and 67% for restaurant predicates. Its mistakes there are confident ones, which follow from a mismatch between the wording of the question and the labelling convention rather than from uncertainty.

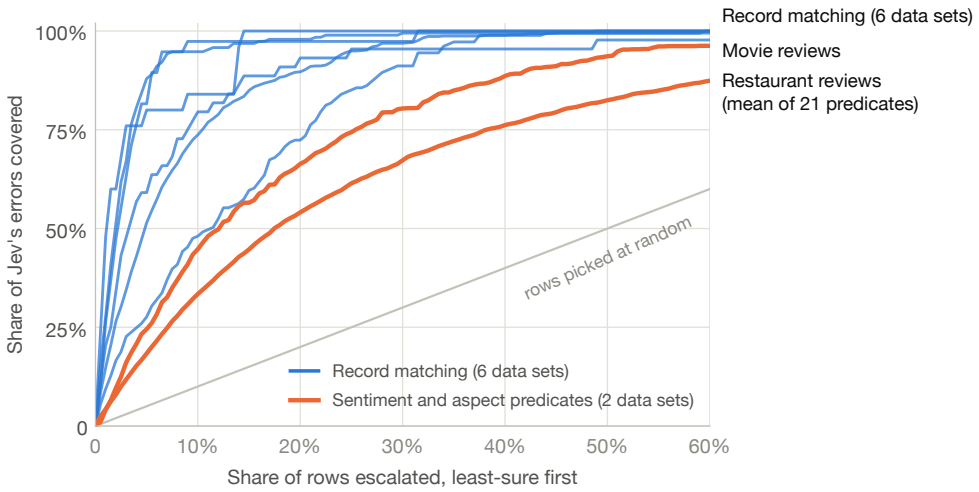


Figure 2. Share of Jev's errors that fall in the rows it is least sure about. This is the most any cascade could recover with a perfect upper tier.

A practical consequence is that this curve can be drawn from a few hundred labels before any expensive model is called, so it can serve as a test of whether a cascade is worth building for a given predicate.

3.3 Semantic aggregates

The task is to estimate the share of positive reviews among 15,388 (truth 81.3%). Without labels every model is biased: Jev thresholded 75.4%, Jev mean probability 70.0%, the LLM on every row 72.9%. Table 3 adds 200 human labels.

Table 3. Estimating a share with 200 labels (2,000 random draws each).

Method	RMSE (points)	95% interval coverage	Label saving vs. sampling
Sampling alone	2.73	95.2%	1.0×
Jev probability + difference correction (primary)	1.72	94.5%	2.5× (theory 2.62×)
Stratified sampling on Jev probability	1.38	95.5%	3.9×
LLM on every row + difference correction	2.18	94.8%	1.6×

The pre-declared 3× was not reached; coverage was. Stratifying on Jev's probability does reach 3.6–4.1×, but it was not the primary estimator. The approach is most useful for GROUP BY: calibrating Jev's probabilities on 200 labels gives a mean error of 3.7 points per movie over 87 movies, against 17.7 points when the same labels are spread across the groups. On the restaurant predicates the gain almost disappears (1.3× with a regression estimator, below 1× with the difference estimator), because Jev's answers correlate less with those labels.

3.4 Is the probability specific to Jev?

No. Table 4 swaps the source of the per-row judgment.

Table 4. Per-row judgment sources on 5,001 movie reviews. Agreement is with the human labels.

Source	Agreement	AUC	Label saving	Cost / 1,000 rows	Latency
Jev probability	91.5%	0.972	2.5×	$0.013	0.29 s
DeepSeek V4.1 Flash token probability	91.5%	0.975	2.2–2.6×	$0.060	0.42 s
DeepSeek V4 Pro token probability	89.3%	0.965	1.8–2.2×	$0.100	0.92 s
Qwen 3.8 Flash token probability	86.7%	0.968	1.9–2.1×	$0.030	2.16 s
GPT 6 Luna, yes/no only	89.6%	n/a	1.5–2.0×	$0.036	2.49 s

On the 21 restaurant predicates mean AUC is 0.871 for Jev and 0.863 for DeepSeek V4.1 Flash. Reading token probabilities from an open model with thinking switched off works as well as Jev; with thinking on, those probabilities collapse to 0 or 1 and are useless. Jev's edge is price and latency.

3.5 Selectivity, ordering and packing

Selectivity. From a 50-row sample, Jev's estimate is within a median factor of 1.13 of the truth on 87 movie subsets (a constant 1/3, PostgreSQL's default for a Boolean function, is off by 2.67). On the 21 restaurant

predicates the median factor is 2.10, above the stop line of 2. Jev estimates how many rows *it* would pass (median factor 1.19), which is enough for costing a plan but not for predicting result size when wording and labels disagree.

No tier to escalate to. Asked one question at a time, the LLM beats Jev by 0.1 points on the restaurant predicates (84.1% against 84.0%), below the 2-point bar, so the escalation-ordering experiment was not built.

Ordering versus packing. Over 1,540 conjunctions of two or three predicates executed by Jev alone, the worst order costs a median 1.11× (two predicates) and 1.27× (three) of the best; ordering by Jev's sampled pass rates is within 1.00×; packing the predicates into one request costs 0.97–1.04× of the best order in tokens with 11% fewer requests, and needs no statistics.

Table 5. One question per request versus 21 questions in one request, 1,000 restaurant sentences.

Model	Accuracy, single	Accuracy, packed	Latency per request	Cost per 1,000 sentences
Jev	84.0%	84.1%	0.29 s to 0.29 s	$0.284 to $0.038
GPT 6 Luna	84.1%	88.0%	2.27 s to 3.51 s	$0.801 to $0.220
DeepSeek V4.1 Flash	86.5%	88.6%	0.45 s to 0.76 s	$1.329 to $0.477

Jev's packed answers agree with its single answers 99.5% of the time at unchanged latency; profiling the whole table took 76 seconds instead of 1,561. Packing is not specific to Jev, however. The LLMs can do it too, and they become 2–4 points *more* accurate, because seeing all the questions helps them separate neighbouring aspects. Packed, Jev remains the cheapest by a factor of 6 to 12.

4 Discussion

Can Jev's strengths be combined with a database? Yes, in the plain sense: it runs as SQL functions, matches a cheap LLM on quality, and costs a third as much.

Can the AI in AI-for-databases be Jev? Mostly not. Supplying statistics for semantic predicates is feasible but worth little: Jev-executed predicates should simply be packed, and there was no clearly better tier whose calls were worth ordering.

Are the claims about cascades right? Only for tasks like record matching. The condition can be checked in advance with a few hundred labels (Figure 2).

The 89× of JEVDB [1] and the 3–5× measured here are not in conflict: most of the difference comes from query rewrites, de-duplication and the baseline's conditions, not from the model.

5 Limitations

Two text data sets beyond matching, and one or two LLMs per role. Published SemBench cost and time figures come from a different model and year; only quality is directly comparable. The movie truth is the critic's verdict rather than the sentiment of the snippet, and the restaurant prompts visibly disagree with the annotation convention (for example "service" against "staff"), so accuracies are low for every model alike. LIMIT queries

return 5–10 rows, so one row moves F1 by 0.1–0.2. LLM timings depend on concurrency and were measured once. Selectivity and ordering were evaluated by replay and not wired into DuckDB's optimizer. All data sets are public and may have been seen in training.

6 Deviations from the pre-declared protocol

- In the qualification test 1–3% of answers from thinking routes were truncated by my token cap; the cap was raised and those rows re-asked. The selection rule was unchanged.
- A metering bug let requests that failed twice go uncounted, so 35 rows across six LLM and cascade results had been computed without an answer. A replay check found and repaired them; mean quality is unchanged to three decimals and Jev's results were unaffected.
- The stratified estimator first used n ± 1 labels; fixing this moved the simulated primary figure from 2.61× to 2.53× (theory 2.62×).
- Added after seeing results, and labelled as such: the top-k-by-probability variants, the packing comparison, and the cascade with a packed upper tier. For packing I had required the LLM's packed accuracy to be within 1 point of its single-question accuracy; it was 2–4 points higher.
- The last experiment was sized to the remaining balance. The platform reserves the full output limit of each in-flight request, which forced three collection runs; no answers were analysed between them.

7 Cost and reproducibility

The whole study cost about $23: $19.97 on OpenRouter (the entire $20 balance) and about $3.40 on Jev. Code, pre-declared criteria and the commands for every table are in the project repository (`experiments/database/README.md`, `experiments/database/`, `src/kevdb/`). The cached judgments are kept locally and are not part of the repository.

8 Proposed next steps

1. **Predict cascade payoff before paying for it.** Test, on six or more further predicate types (extraction checks, topic, toxicity, relevance), whether the error-concentration curve from 200 labels predicts the measured payoff. Roughly $5 of Jev and $30–60 of a strong model.
2. **Make Jev's errors findable.** On the restaurant predicates, test whether recalibration on 200 labels, single-choice formulations that encode mutually exclusive aspects, or wording taken from the annotation guideline concentrate the errors. Jev only, under $5.
3. **Put the two model-agnostic rewrites into the optimizer.** Implement join decomposition and predicate packing as DuckDB rewrite rules and evaluate on all text-only SemBench queries, with LOTUS and Palimpzest run locally on the same LLM for a like-for-like comparison. Roughly $5 of Jev and $20–40 of LLM baselines.

References

1. Z. Wang, H. Yan, F. Zhao, C. Liu. *Prune First, Decide Fast: Scalable Semantic Query Processing with JEVDB*. arXiv:2610.02046, 2026.

2. *SemBench: A Benchmark for Semantic Query Processing Engines.* arXiv:2511.01716, 2025.
3. L. Patel et al. *Semantic Operators: A Declarative Model for Rich, AI-based Data Processing* (LOTUS). arXiv:2407.11418, 2024.
4. Q. Jiang et al. *A Challenge Dataset and Effective Models for Aspect-Based Sentiment Analysis* (MAMS). EMNLP 2019.
5. Y. Li et al. *Deep Entity Matching with Pre-Trained Language Models* (Ditto; Magellan benchmark splits). arXiv:2004.00584, 2020.
6. A. Angelopoulos et al. *Prediction-Powered Inference.* Science, 2023.